



UNIVERSIDADE KIMPA VITA
ESCOLA SUPERIOR POLITÉCNICA DO UÍGE
CURSO DE LICENCIATURA EM ENGENHARIA INFORMÁTICA

**RELATÓRIO CIENTÍFICO DO TRABALHO EM
GRUPO DE LINGUAGEM DE PROGRAMAÇÃO**

Sistema de Gestão Hospitalar Inteligente

3º Ano
Turma: Pavilhão
Período: Manhã
Curso: Engenharia Informática

Orientador

Moyo Kanivengidio, Msc

MAIO DE 2026

ENTEGRANTES DO GRUPO

1. António Lopes Tamba
2. Jeremias Manuel
3. Lukombo David
4. Maria David Kiala
5. Rosa Sozinho Garcia

1. RESUMO

O presente relatório descreve o desenvolvimento e implementação do Sistema Hospitalar Inteligente, uma aplicação desktop desenvolvida em Python para a gestão integrada de serviços hospitalares. O sistema incorpora módulos de gestão de pacientes, médicos, consultas, utilizadores e triagem com suporte de Inteligência Artificial (IA), utilizando a API do **Google Gemini** para análise de sintomas e sugestão de níveis de urgência.

A aplicação foi construída com Tkinter para a interface gráfica, e Microsoft SQL Server como sistema de gerenciamento de Base de Dados relacional. O sistema implementa controlo de acesso baseado em perfis (RBAC), autenticação por **hash SHA-256** e uma arquitetura em camadas que separa a lógica de negócio, dados e apresentação.

Os resultados demonstram que o sistema satisfaz os requisitos funcionais de um ambiente hospitalar real, proporcionando uma interface intuitiva, segurança de dados e a integração inovadora de IA para apoio à decisão clínica, contribuindo para a modernização dos serviços de saúde em Angola.

2. INTRODUÇÃO

1.1 Contextualização

Os sistemas de informação hospitalares desempenham um papel fundamental na modernização dos serviços de saúde. Em Angola, e em particular no Uíge, a crescente demanda por serviços de saúde de qualidade exige soluções tecnológicas que otimizem o fluxo de trabalho clínico, reduzam erros administrativos e melhorem a experiência tanto dos profissionais de saúde como dos pacientes.

O Hospital Geral no Uíge, referência do sistema de saúde angolano, enfrenta desafios comuns a instituições de saúde em desenvolvimento: Gestão Manual de Registos, dificuldade de acesso rápido a históricos de pacientes e ausência de ferramentas de apoio à decisão clínica. O Sistema Hospitalar Inteligente surge como resposta directa a estas necessidades.

1.2 OBJETIVOS

1.2.1 Objectivo Geral

Desenvolver um sistema de informação hospitalar completo, seguro e inteligente, que integre a gestão de pacientes, médicos e consultas com uma funcionalidade de triagem assistida por Inteligência Artificial.

1.2.2 Objectivos Específicos

- Implementar módulos de gestão de pacientes, médicos, consultas e utilizadores;
- Desenvolver um sistema de autenticação seguro com controlo de acesso por perfis;
- Integrar a API do Google Gemini para triagem inteligente de pacientes;
- Criar uma interface gráfica moderna, responsiva e de fácil utilização;
- Garantir a persistência e integridade dos dados através do SQL Server;
- Implementar relatórios e visualizações estatísticas para apoio à gestão.

1.3 Justificativa

A digitalização dos processos hospitalares reduz significativamente erros de comunicação, acelera o atendimento e permite uma visão integrada do histórico clínico dos pacientes. A integração de IA na triagem representa um avanço que auxilia os profissionais de saúde a priorizar casos críticos, potencialmente salvando vidas em contextos de alta demanda.

3. INTELIGÊNCIA ARTIFICIAL NA SAÚDE

A aplicação de Inteligência Artificial na medicina tem crescido exponencialmente. No contexto da triagem médica, modelos de linguagem de grande escala como o **Google Gemini** demonstram capacidade de analisar sintomas descritos em linguagem natural e sugerir níveis de urgência com precisão comparável a profissionais treinados em triagem inicial.

O sistema utiliza o modelo gemini-2.0-flash através da API REST do Google, enviando dados estruturados do paciente (nome, idade, género, sintomas, duração) e recebendo uma avaliação em língua portuguesa com nível de urgência (CRÍTICO, URGENTE, MODERADO, NÃO URGENTE), possíveis causas, encaminhamento recomendado e acções imediatas.

2.3 Tecnologias Utilizadas

Tecnologia	Versão / Tipo	Função no Sistema
Python	3.14	Linguagem principal de desenvolvimento
Tkinter	Última estável	Interface gráfica moderna (tema escuro/claro)
SQLAlchemy	ORM	Mapeamento objecto-relacional e gestão de sessões
SQL Server	2026	Base de dados relacional principal
pyodbc / ODBC 17	Driver	Conector entre Python e SQL Server
Google Gemini API	gemini-2.0-flash	Motor de IA para triagem de pacientes
Matplotlib / Pandas	Análise	Geração de gráficos e relatórios estatísticos
Pillow (PIL)	Imagens	Processamento de imagens na interface
hashlib SHA-256	Segurança	Encriptação de senhas dos utilizadores

3. METODOLOGIA

3.1 Abordagem de Desenvolvimento

O projecto seguiu uma abordagem de desenvolvimento iterativo e incremental, com fases bem definidas: levantamento de requisitos, desenho da arquitectura, implementação por módulos, testes e refinamento. Esta abordagem permitiu identificar e corrigir problemas progressivamente, garantindo a qualidade do produto final.

3.2 Arquitectura do Sistema

O sistema adopta uma arquitectura em camadas com separação clara de responsabilidades:

Camada	Pasta	Responsabilidade
Apresentação	gui/	Interface gráfica: login, dashboard, módulos CRUD
Lógica de Negócio	services/ e utils/	Autenticação, permissões, relatórios, triagem IA
Acesso a Dados	models/	Modelos ORM: Paciente, Médico, Consulta, Utilizador
Base de Dados	database/	Conexão, criação de tabelas, migrações
IA / Externo	ai/ e gui/triagem_ia.py	Integração com Google Gemini API

3.3 Modelo de Dados

A base de dados HospitalInteligente no SQL Server contém as seguintes tabelas principais:

Tabela	Campos Principais	Relações
utilizadores	id, nome, email, senha, perfil, ativo, criado_em	1:N com consultas (médico)
pacientes	id, nome, numero_processo, género, telefone, data_nascimento	1:N com consultas
medicos	id, nome, especialidade, telefone, email, numero_ordem, activo	Tabela independente
consultas	id, paciente_id, medico_id, data, hora, estado, motivo	N:1 paciente, N:1 médico

3.4 Sistema de Autenticação e Permissões

A segurança é implementada em dois níveis. A autenticação verifica email e senha (hash SHA-256) na tabela utilizadores. As permissões são controladas por um sistema RBAC (Role-Based Access Control) com três perfis:

Módulo	Administrador	Médico	Recepção
Início / Dashboard	✓	✓	✓
Pacientes	✓	✓	✓
Médicos	✓	X	X
Utilizadores	✓	X	X
Consultas	✓	✓	✓
Triagem IA	✓	✓	X
Relatórios	✓	✓	X

4. DESENVOLVIMENTO E IMPLEMENTAÇÃO

4.1 Estrutura do Projecto

O projecto está organizado da seguinte forma:

Ficheiro / Pasta	Descrição
main.py	Ponto de entrada da aplicação; inicializa tema e janela de login
gui/login.py	Janela de autenticação com validação de credenciais
gui/dashboard.py	Dashboard principal com sidebar de navegação e cards de estatísticas
gui/pacientes.py	Módulo CRUD completo para gestão de pacientes
gui/medicos.py	Módulo CRUD completo para gestão de médicos
gui/consultas.py	Módulo de agendamento e gestão de consultas
gui/utilizadores.py	Módulo de gestão de utilizadores do sistema
gui/triagem_ia.py	Interface de triagem com integração Google Gemini
gui/relatorios.py	Módulo de relatórios estatísticos com gráficos
models/*.py	Modelos SQLAlchemy: Paciente, Médico, Consulta, Utilizador
database/conexao.py	Configuração da ligação ao SQL Server via pyodbc
utils/autenticacao.py	Lógica de autenticação com hash SHA-256
utils/permissoes.py	Sistema RBAC com dicionário de permissões por perfil
utils/context_managers.py	Gestor de contexto para sessões da base de dados

4.2 Interface Gráfica

A interface utiliza Tkinter com tema escuro e paleta de cores azul corporativa, proporcionando uma aparência moderna e profissional. O dashboard apresenta uma barra lateral (sidebar) com navegação por módulos, indicadores visuais de permissão (cadeado para módulos bloqueados) e cards de estatísticas em tempo real.

Todos os módulos CRUD implementam um padrão consistente: tabela de listagem com pesquisa em tempo real, formulário de criação/edição em janela modal, confirmação para operações destrutivas e feedback visual ao utilizador.

4.3 Módulo de Triagem com IA

O módulo de triagem integra a **API REST do Google Gemini** (modelo gemini-2.0-flash) para análise de sintomas em linguagem natural. O fluxo de funcionamento é:

- O utilizador preenche dados do paciente (nome, idade, género, sintomas, duração)
- A aplicação constrói um prompt estruturado em Português de Angola
- O pedido é enviado via HTTP para a **API do Gemini** com a chave API do utilizador
- A resposta é exibida com nível de urgência, causas prováveis e recomendações
- O processamento é executado em thread separada para não bloquear a interface

Justificativa da Não Implementação da Análise Automática de Triagem por IA

Embora o projeto tenha sido concebido com a possibilidade de integração de Inteligência Artificial (IA) para apoio à triagem hospitalar, a funcionalidade de análise automática dos pacientes não foi implementada na versão atual do sistema devido a limitações técnicas, científicas e operacionais.

Em primeiro lugar, os modelos de IA utilizados em ambientes clínicos exigem grandes volumes de dados médicos reais, devidamente classificados e validados por profissionais de saúde. Como o presente projeto possui carácter académico e não dispõe de acesso a bases de dados hospitalares certificadas, não seria possível treinar um modelo com o nível de precisão e confiabilidade exigido para apoiar decisões médicas.

Além disso, a implementação de um sistema de triagem baseado em IA envolve questões éticas e legais relacionadas à segurança dos pacientes. Um diagnóstico ou classificação incorreta poderia comprometer o atendimento clínico, sendo necessária a validação por especialistas médicos e por entidades reguladoras antes da sua utilização em ambiente real.

4.4 Gestão da Base de Dados

A ligação ao SQL Server é configurada em `database/conexao.py` com autenticação Windows integrada (`trusted_connection=yes`), eliminando a necessidade de gestão de credenciais de BD separadas. O SQLAlchemy é utilizado como ORM, com o gestor de contexto `sessao_db()` garantindo que todas as sessões são correctamente abertas, commitadas e fechadas, prevenindo fugas de recursos.

5. RESULTADOS E DISCUSSÃO

5.1 Funcionalidades Implementadas

O sistema foi desenvolvido e testado com sucesso, implementando todas as funcionalidades planeadas:

Módulo	Estado	Observações
Autenticação e Login	Completo	Hash SHA-256, validação de utilizador activo
Dashboard com Estatísticas	Completo	Cards em tempo real, foto do hospital no fundo
Gestão de Pacientes	Completo	CRUD completo, pesquisa, número de processo único
Gestão de Médicos	Completo	CRUD completo, activar/desactivar, nº de ordem
Gestão de Consultas	Completo	Agendamento, estados, associação paciente-médico
Gestão de Utilizadores	Completo	Perfis admin/médico/recepção, RBAC
Triagem com IA	Completo	Google Gemini, análise de sintomas, urgência
Relatórios Estatísticos	Completo	Gráficos matplotlib, exportação
Controlo de Permissões	Completo	RBAC, módulos bloqueados com cadeado visual

5.2 Análise de Desempenho

Os testes realizados demonstraram que o sistema responde adequadamente em condições normais de utilização. A interface gráfica mantém fluidez mesmo durante operações de base de dados.

5.3 Limitações e Trabalho Futuro

As principais limitações identificadas incluem:

- Dependência de ligação à internet para a funcionalidade de triagem com IA;
- A configuração da base de dados está ligada a um servidor específico (DESKTOP-QSQRSO8), necessitando adaptação para outros ambientes;

- Ausência de módulo de prescrições e histórico clínico detalhado;
- Falta de integração com equipamentos médicos ou outros sistemas HIS.

Como trabalho futuro, propõe-se:

- Migração para arquitectura cliente-servidor para suporte multi-utilizador simultâneo;
- Implementação de módulo de farmácia e gestão de medicamentos;
- Desenvolvimento de versão web ou móvel para acesso remoto;
- Implementação de IA local (offline) para triagem sem internet.

6. CONCLUSÃO

O Sistema Hospitalar Inteligente representa uma contribuição significativa para a modernização dos serviços de saúde em Angola. O projecto demonstrou ser possível desenvolver uma solução tecnológica completa e funcional, integrando tecnologias modernas como Python, SQLAlchemy, SQL Server e Inteligência Artificial, adaptada ao contexto hospitalar angolano.

A integração do Google Gemini para triagem médica é um diferencial inovador que posiciona o sistema na vanguarda das soluções de saúde digitais, proporcionando aos profissionais de saúde uma ferramenta de apoio à decisão que pode melhorar a qualidade e rapidez do atendimento.

O sistema atinge os objectivos propostos, implementando todas as funcionalidades essenciais de um HIS moderno, com uma interface intuitiva, segurança robusta e arquitectura escalável. As limitações identificadas representam oportunidades claras de evolução para versões futuras do produto.

Assim concluímos que, o desenvolvimento deste projecto não só produziu uma ferramenta útil para o contexto hospitalar, como também aprofundou as competências técnicas da equipa nas áreas de desenvolvimento de software, bases de dados relacionais, interfaces gráficas e integração de serviços de Inteligência Artificial.

7. Referências Bibliográficas

- [1] Python Software Foundation. (2024). Python 3 Documentation. <https://docs.python.org/3/>
- [2] SQLAlchemy Authors. (2024). SQLAlchemy Documentation. <https://docs.sqlalchemy.org/>
- [3] Tom Schimansky. (2024). CustomTkinter — Modern Tkinter UI-Library. <https://customtkinter.tomschimansky.com/>
- [4] Google LLC. (2024). Gemini API Documentation. <https://ai.google.dev/docs>
- [5] Microsoft Corporation. (2024). SQL Server Documentation. <https://learn.microsoft.com/en-us/sql/>
- [6] World Health Organization. (2022). Digital Health Strategy 2020-2025. WHO Press.
- [7] Shortliffe, E. H., & Cimino, J. J. (2021). Biomedical Informatics: Computer Applications in Health Care and Biomedicine (5th ed.). Springer.
- [8] Topol, E. J. (2019). High-performance medicine: the convergence of human and artificial intelligence. *Nature Medicine*, 2